Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:51 Uhr

ANHANG

B

Datentypen in Java

Java kennt zwei Arten von Typen: primitive Typen und Objekttypen.

Primitive Typen werden direkt in Variablen gespeichert und haben Wertsemantik (Werte werden bei einer Zuweisung an eine andere Variable kopiert). Primitive Typen haben nichts mit Klassen zu tun und verfügen nicht über Methoden.

Objekttypen werden als Referenzen auf Objekte gespeichert (nicht die Objekte selbst). Bei einer Zuweisung an eine andere Variable wird nur die Referenz kopiert, nicht das Objekt. (Sie erfahren mehr über die Referenzsemantik im Abschnitt B.3.)

B.1 Primitive Typen

Die folgende Tabelle führt alle primitiven Typen der Sprache Java auf:

Typname	Beschreibung	Beispielkonstanten		
Ganze Zahlen:				
`byte`	ganze Zahl in 8 Bit	`24`	`-2`	
`short`	ganze Zahl in 16 Bit	`137`	`-119`	
`int`	ganze Zahl in 32 Bit	`5409`	`-2003`	
`long`	ganze Zahl in 64 Bit	`423266353`	`55L`	
Gleitpunktzahlen:				
`float`	einfache Fließkommazahl (32 Bit)	`43.889F`		
`double`	doppelte Fließkommazahl (64 Bit)	`42.63`	`2.4e5`	
Andere Typen:				
`char`	einzelnes Zeichen (16 Bit)	`'m'`	`'?'`	`'\u00F6'`
`boolean`	ein boolescher Wert (wahr oder falsch)	`true`	`false`	

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:51 Uhr

Hinweise:

- Eine Zahl ohne Dezimalpunkt wird üblicherweise als ein `int` aufgefasst, aber automatisch konvertiert, wenn sie einer Variablen vom Typ `byte`, `short` oder `long` zugewiesen wird (wenn der Wert passt). Sie können eine Konstante als `long` deklarieren, indem Sie ein `L` an die Zahl anhängen (auch `l` – ein kleines `L` – kann benutzt werden, sollte aber wegen der Verwechslungsgefahr mit der Eins vermieden werden).
- Eine Zahl mit einem Dezimalpunkt ist vom Typ `double`. Sie können eine `float`-Konstante definieren, indem Sie ein `f` oder ein `F` an die Zahl anhängen.
- Ein Zeichen kann als Unicode-Zeichen in einfachen Anführungsstrichen oder als Unicode-Wert mit einem vorangestellten `\u` angegeben werden.
- Die beiden booleschen Konstanten sind `true` und `false`.

Da Variablen mit primitivem Typ nicht auf Objekte verweisen, stehen für die primitiven Typen keine Methoden zur Verfügung. Wenn jedoch in bestimmten Zusammenhängen ein Objekttyp benötigt wird, kann über Autoboxing ein primitiver Wert automatisch in ein korrespondierendes Objekt umgewandelt werden. In Anhang B.4 finden Sie mehr Informationen dazu.

Die folgende Tabelle gibt die verfügbaren Minimal- und Maximalwerte der numerischen Typen an.

Typ	Minimum	Maximum
`byte`	`-128`	`127`
`short`	`-32768`	`32767`
`int`	`-2147483648`	`2147483647`
`long`	`-9223372036854775808`	`9223372036854775807`
	Positives Minimum	**Positives Maximum**
`float`	`1.4e-45`	`3.4028235e38`
`double`	`4.9e-324`	`1.7976931348623157e308`

B.2 Cast-Operator für primitive Typen

Manchmal ist es erforderlich, den Wert eines primitiven Typs in einen Wert eines anderen primitiven Typs umzuwandeln. Meist soll dabei ein Wert mit einem bestimmten Wertebereich in einen Wert mit einem kleineren Wertebereich umgewandelt werden. Dies wird auch als *Casting* bezeichnet. Casting geht fast immer einher mit einem Verlust an Informationen, wie es zum Beispiel der Fall ist, wenn wir einen Fließkommatyp in einen ganzzahligen Typ umwandeln. Java erlaubt die Typumwandlung zwischen allen numerischen Datentypen. Allerdings ist es nicht möglich, einen booleschen Wert durch Casting in einen anderen Datentyp umzuwandeln oder umgekehrt.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:51 Uhr

Der Cast-Operator besteht aus dem in runden Klammern gesetzten Namen eines primitiven Datentyps und steht vor einer Variablen oder einem Ausdruck. Zum Beispiel

```
int wert = (int) durchschnitt;
```

Wenn **durchschnitt** eine Variable vom Datentyp **double** ist und den Wert 3,9 enthält, würde die obige Anweisung in der Variablen **wert** den ganzzahligen Wert 3 speichern (Umwandlung durch Abrundung).

B.3 Objekttypen

Alle Typen, die nicht im Abschnitt B.1 aufgeführt sind, sind Objekttypen. Dies schließt Klassen- und Interface-Typen aus der Standardbibliothek von Java (wie **String**) ebenso ein wie benutzerdefinierte Typen.

Eine Variable eines Objekttyps hält eine Referenz (oder einen „Zeiger") auf ein Objekt. Zuweisung und Parameterübergabe haben Referenzsemantik (es wird die Referenz kopiert, nicht das Objekt). Nach Zuweisung einer Variablen an eine andere verweisen beide auf dasselbe Objekt. Man sagt, die beiden Variablen sind Aliase für dasselbe Objekt. Diese Regel gilt nicht nur für eine einfache Zuweisung zwischen Variablen, sondern auch, wenn ein Objekt als aktueller Parameter an eine Methode übergeben wird. Folglich bleiben Zustandsänderungen an einem Objekt über einen formalen Parameter nach Beenden der Methode in dem aktuellen Parameter bestehen.

Klassen sind Schablonen für Objekte, die die Datenfelder und Methoden definieren, die jede Instanz besitzt.

Arrays verhalten sich wie Objekttypen – auch sie haben Referenzsemantik. Es gibt keine Klassendefinition.

B.4 Wrapper-Klassen

Zu jedem primitiven Typ in Java gibt es eine korrespondierende Wrapper-Klasse, die den jeweiligen primitiven Typ als Objekttyp repräsentiert. Dies ermöglicht den Einsatz primitiver Typen auch an Stellen, an denen Objekttypen erwartet werden, indem das sogenannte *Autoboxing* zur Anwendung kommt. Die folgende Tabelle benennt die primitiven Typen und die zugeordneten Wrapper-Klassen aus dem Paket **java.lang**. Abgesehen von **Integer** und **Character** entsprechen die Namen der Wrapper-Klassen den Namen der primitiven Typen mit einem großen Anfangsbuchstaben.

Immer wenn ein primitiver Typ in einem Kontext verwendet wird, der einen Objekttyp erfordert, wendet der Compiler Autoboxing an, indem er automatisch den primitiven Wert in ein entsprechendes Wrapper-Objekt einbettet. Dies bewirkt beispielsweise, dass ein primitiver Wert direkt in eine Sammlung eingefügt werden kann. Die inverse Operation – *Unboxing* – wird ebenfalls automatisch angewendet, wenn ein Wrapper-Objekt in einem Kontext verwendet wird, der den Wert des entsprechenden primitiven Typs erwartet.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:51 Uhr

Primitiver Typ	Wrapper-Typ
`byte`	`Byte`
`short`	`Short`
`int`	`Integer`
`long`	`Long`
`float`	`Float`
`double`	`Double`
`char`	`Character`
`boolean`	`Boolean`

B.5 Cast-Operator für Objekttypen

Da viele Objekte Teil einer Vererbungshierarchie von Objekttypen sind, ist es manchmal notwendig, die Objektreferenz eines Datentyps in eine Referenz eines Subtyps tiefer in der Vererbungshierarchie umzuwandeln. Dieser Prozess wird auch als *Casting* (oder *Downcasting*) bezeichnet. Der zugehörige Cast-Operator besteht aus dem in runden Klammern gesetzten Namen des Zieltyps (eine Klasse oder ein Interface) und steht vor einer Variablen oder einem Ausdruck. Zum Beispiel

```
Auto a = (Auto) kfz;
```

Wenn der deklarierte (d.h. statische) Typ der Variablen **`kfz`** vom Typ **`Kraftfahrzeug`** ist und **`Auto`** eine Subklasse von **`Kraftfahrzeug`** ist, wird sich diese Anweisung kompilieren lassen. Zur Laufzeit wird zusätzlich geprüft, ob das Objekt, auf das sich **`kfz`** bezieht, wirklich ein **`Auto`** ist und nicht eine Instanz eines anderen Subtyps.

Beachten Sie unbedingt, dass das Casting zwischen Objekttypen etwas völlig anderes ist als das Casting zwischen primitiven Typen (Anhang B.2). Insbesondere bewirkt das Casting zwischen Objekttypen keine Änderung des betreffenden Objekts. Es geht allein darum, Zugriff auf Typinformationen zu erhalten, die bereits Teil des Objekts sind, weil sie zu seinem dynamischen Typ gehören.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:51 Uhr

ANHANG

C Operatoren

C.1 Arithmetische Ausdrücke

Java bietet zahlreiche Operatoren für arithmetische und logische Ausdrücke. Tabelle C.1 zeigt alle Konstrukte, die als Operatoren eingestuft werden, einschließlich dem Cast-Operator und der Parameterübergabe. Bei den meisten Operatoren handelt es sich entweder um binäre Operatoren (deren Operanden links und rechts stehen) oder um unäre Operatoren (die nur einen Operanden haben). Die wichtigsten binären arithmetischen Operatoren lauten:

- `+` Addition
- `-` Subtraktion
- `*` Multiplikation
- `/` Division
- `%` Modulo oder ganzzahliger Rest bei Division

Die Ergebnisse der Operationen Division und Modulo hängen davon ab, ob ihre Operanden ganze Zahlen oder Fließkommazahlen sind. Bei einer Division zwischen zwei `int`-Werten wird ein `int`-Ergebnis ohne Rest geliefert, aber zwischen Fließkommawerten wird ein Fließkommawert geliefert:

- `5 / 3` liefert als Ergebnis `1`
- `5.0 / 3` liefert als Ergebnis `1.66666666666666667`

(Beachten Sie, dass nur einer der Operanden eine Fließkommazahl sein muss, um zu einem Fließkommaergebnis zu führen.)

Wenn in einem arithmetischen Ausdruck mehr als ein Operator enthalten ist, werden für die Auswertungsreihenfolge *Prioritätsregeln* für Operatoren angewendet. In Tabelle C.1 erscheinen Operatoren mit höherer Priorität weiter oben; wir sehen also beispielsweise, dass Multiplikation, Division und Modulo eine höhere Priorität haben als die Addition und die Subtraktion. Die folgenden Ausdrücke liefern deshalb beide das Ergebnis `100`:

```
51 * 3 - 53
154 - 2 * 27
```

Binäre Operatoren mit derselben Priorität werden von links nach rechts ausgewertet, während unäre Operatoren mit derselben Priorität von rechts nach links ausgewertet werden.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 10:51 Uhr